

Task 2 — Service Configuration Management

In this task, we need to implement the **3 things**,

1. `application.yml`
2. Environment variables
3. Profiles

We'll do this step-by-step in **Eclipse**.

Step 1: Create `application.yml`

For your **Order Service**:

Go to:

`order-service`

→ `src/main/resources`

→ `application.yml`

If `application.yml` doesn't exist:

Right-click `src/main/resources` → New → File → type `application.yml` → Finish

Put this inside:

`spring:`

`application:`

`name: order-service`

`datasource:`

`url: jdbc:mysql://localhost:3306/order_db`

`username: ${DB_USERNAME:root}`

password: \${DB_PASSWORD:root}

driver-class-name: com.mysql.cj.jdbc.Driver

jpa:

hibernate:

ddl-auto: update

show-sql: true

server:

port: 8083

- Service name → order-service
- Database → order_db
- Username → taken from DB_USERNAME
- Password → taken from DB_PASSWORD
- Port → **8083**
- \${DB_USERNAME:root} means:
 - o use the environment variable if available
 - o otherwise use root

Step 2: Create environment variables

We don't want to hard-code passwords in the application.

In Windows:

Search → Edit the system environment variables → Environment Variables

Under **User variables**, click **New**.

Create:

Variable name: DB_USERNAME

Variable value: root

Create another:

Variable name: DB_PASSWORD

Variable value: your_mysql_password

For example:

DB_USERNAME = root

DB_PASSWORD = root

Use the actual password you created for your MySQL installation.

Step 3: Create Spring profiles

Create these files inside:

Src/main/resources

Create:

application-dev.yml

application-prod.yml

application-dev.yml

server:

port: 8083

spring:

datasource:

url: jdbc:mysql://localhost:3306/order_db

application-prod.yml

server:

port: 8083

spring:

datasource:

url: jdbc:mysql://localhost:3306/order_db

For a real production application, the production database credentials should come from environment variables or a secrets manager rather than being written directly into the file.

Step 4: Activate the profile

In `application.yml`, add:

`spring:`

`profiles:`

`active: dev`

So, your complete beginning becomes:

`spring:`

`application:`

`name: order-service`

`profiles:`

`active: dev`

`datasource:`

`url: jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3306}/${DB_NAME:order_db}`

`username: ${DB_USERNAME:root}`

`password: ${DB_PASSWORD:root}`

`driver-class-name: com.mysql.cj.jdbc.Driver`

```
jpa:  
  hibernate:  
    ddl-auto: update  
  show-sql: true
```

```
server:  
  port: ${SERVER_PORT:8083}
```

Now the application will use the **dev profile**.

Step 5 — External Configuration

External configuration means configuration can be supplied **outside your Java code/JAR**.

For example, you can create a folder:

```
C:\order-config
```

And create

```
Application.yml
```

inside it.

You can then start your application with an external configuration location.

In Eclipse:

Right-click Order Service → Run As → Run Configurations

Go to:

Arguments → VM arguments

Add:

```
-Dspring.config.additional-location=file:C:/order-config/
```

Click:

Apply → Run

Spring Boot will read configuration from the external location.

Step 6 — Secrets Management Basics

Never put real passwords such as:

```
password: MyRealPassword123
```

inside your Git repository.

Instead use:

```
password: ${DB_PASSWORD}
```

For example:

```
spring:
```

```
  datasource:
```

```
    username: ${DB_USERNAME}
```

```
    password: ${DB_PASSWORD}
```

Then the password comes from the environment.

For a real production application, secrets should preferably be stored in a dedicated secret-management system such as **AWS Secrets Manager**, **Azure Key Vault**, **HashiCorp Vault**, or your organization's approved secrets platform.

Step 7 — Final Folder Structure

Your Order Service should now look like:

order-service

```
|
|
|└─ src
|
|  └─ main
|
|    └─ java
|
|      └─ com.example.orderservice
|
|        |
|
|        └─ resources
|
|          └─ application.yml
|
|          └─ application-dev.yml
|
|          └─ application-prod.yml
|
|
└─ pom.xml
```

Task 2 is now complete.

